Caching Strategies Caching is the practice of storing a copy of expensive-to-compute or expensive-to-fetch data somewhere faster to access, so that a repeated request for the same data can be served without repeating the original work. Nearly every performance problem in a web application eventually gets addressed, at least partially, by caching something: a database query result, a rendered page fragment, a computed aggregate, or the response of a slow external service call. Redis is the most common choice for an external cache in modern backend systems. It is an in-memory data store that supports simple key-value gets and sets as well as richer structures like lists, sets, sorted sets, and hashes, all with sub-millisecond latency under normal conditions. Because Redis runs as a separate process from the application, a cache stored there is shared across every application instance, which matters as soon as a service runs behind a load balancer with more than one replica. Cache eviction policy determines what happens when the cache is full and a new item needs to be stored. Least recently used, commonly abbreviated LRU, evicts whichever entry has gone the longest without being read, on the theory that data accessed recently is likely to be accessed again soon. Least frequently used instead tracks access counts and evicts the least popular entry. Time-based expiry, or a time to live on each entry, is often layered on top of either policy so that stale data cannot remain cached indefinitely even if it keeps getting accessed. Cache invalidation is famously one of the two hard problems in computer science, and for good reason. Keeping cached data consistent with its source of truth requires either expiring entries aggressively enough that staleness never matters, or actively pushing invalidation events whenever the underlying data changes. Write-through caching updates the cache synchronously whenever the source data is written, keeping the two in lockstep at the cost of extra latency on writes. Write-behind caching updates the cache immediately but defers the write to the source of truth, trading a small consistency window for lower write latency. A cache stampede occurs when a popular cached entry expires and a large number of concurrent requests all miss the cache at once, each independently trying to regenerate the same expensive value and hammering the backing store simultaneously. Common mitigations include locking so only one request regenerates the value while others wait, or probabilistic early expiration so that entries refresh slightly before they actually expire, spreading the regeneration load out over time instead of letting it land all at once.